

RELATÓRIO DE PROCESSO DE SOFTWARE

Projeto Coleta Premiada

Documento DOC-6

Equipe: Luis, Rangel, Heitor (Scrum Master) e Enzo

8 de julho de 2026

Sumário

1. Identificação do Grupo
2. Processo de Software Adotado
3. Organização das Tarefas
4. Condução das Sprints e Iterações
5. Cerimônias e Comunicação
6. Priorização dos Requisitos
7. Dificuldades Encontradas
8. Como as Dificuldades Foram Superadas
9. Métricas de Progresso
10. Retrospectiva Final
11. Conclusão

1. Identificação do Grupo

O projeto Coleta Premiada foi desenvolvido pelo grupo composto por Luis, Rangel, Heitor e Enzo. Durante o desenvolvimento, Heitor foi considerado o Scrum Master do grupo, atuando como apoio na organização das demandas, alinhamento entre integrantes e acompanhamento do andamento geral das tarefas.

O trabalho foi dividido entre diferentes repositórios do ecossistema:

- Coleta-Premiada: backend principal/Core em Django.
- cp-collection-ms: microsserviço de coleta.
- coleta-premiada-frontend: aplicação frontend.
- coleta-observability: observabilidade com Prometheus e Grafana.

2. Processo de Software Adotado

O grupo adotou um processo híbrido, mesclando práticas de Scrum e Kanban. A escolha ocorreu porque o grupo precisava de organização por tarefas e acompanhamento visual do progresso, mas não possuía uma rotina fixa de reuniões em dias pré-definidos.

Na prática, o Scrum foi utilizado como referência para organizar o trabalho em ciclos de entrega, discutir prioridades, revisar o que estava em andamento e realizar alinhamentos entre os integrantes. Já o Kanban foi usado para visualizar o fluxo das atividades por meio de um quadro no GitHub Projects.

O processo real utilizado foi adaptativo: as reuniões e alinhamentos aconteciam conforme surgiam demandas, bloqueios ou oportunidades de encontro. Esses alinhamentos ocorreram principalmente por WhatsApp, reuniões on-line e encontros presenciais.

3. Organização das Tarefas

Para organizar as demandas, foi criado um projeto no GitHub com um quadro Kanban contendo as seguintes colunas:

- Backlog
- Ready
- In Progress
- In Review
- Done

As tarefas eram criadas como issues e movidas no quadro conforme o progresso. O Backlog concentrava ideias e demandas ainda não iniciadas. A coluna Ready representava tarefas prontas para desenvolvimento. A coluna In Progress indicava tarefas em execução. A coluna In Review era usada para atividades em revisão ou aguardando validação. A coluna Done representava tarefas concluídas.

Conforme o projeto progredia, o grupo decidia coletivamente quais tarefas cada integrante assumiria. Essas decisões eram feitas a partir da necessidade do projeto, da disponibilidade dos membros e da área de maior afinidade ou contexto técnico de cada um.

4. Condução das Sprints e Iterações

Embora o grupo tenha usado Scrum como referência, as sprints não seguiram um calendário rígido com datas fixas de início, fim e reuniões obrigatórias. O trabalho foi conduzido em iterações naturais, acompanhando as entregas necessárias em cada etapa do projeto.

No início, o foco principal foi o planejamento da arquitetura e a organização do backend. Essa decisão foi importante porque o backend concentrava as regras de negócio principais, integrações com banco de dados, mensageria, autenticação, auditoria e comunicação com o microserviço.

Depois da base arquitetural, as iterações passaram a incluir:

- implementação de regras de negócio do programa de pontos;
- integração entre Core e microserviço por RabbitMQ;
- desenvolvimento do microserviço de coleta;
- criação do frontend;
- autenticação e telas principais;
- auditoria e logs;
- monitoramento e observabilidade;
- backup e restore;
- documentação técnica e arquitetural.

As prioridades eram revisadas conforme o grupo identificava novas dependências ou dificuldades. Dessa forma, o processo foi mais flexível do que um Scrum tradicional, mas manteve uma estrutura de acompanhamento baseada em tarefas, revisões e entregas incrementais.

5. Cerimônias e Comunicação

As cerimônias foram adaptadas à realidade do grupo. Não houve uma agenda formal fixa para daily, planning, review e retrospectiva em todos os ciclos. Em vez disso, os alinhamentos aconteciam quando havia necessidade de decisão ou quando algum integrante encontrava bloqueios.

As principais formas de comunicação foram:

- grupo no WhatsApp;
- reuniões on-line;
- conversas presenciais;
- comentários e organização por issues e pull requests no GitHub.

As reuniões eram usadas para discutir prioridades, dividir tarefas, resolver dúvidas de integração e decidir próximos passos. O WhatsApp foi o canal mais rápido para comunicação diária e ajustes pontuais.

6. Priorização dos Requisitos

Os requisitos foram priorizados considerando dependências técnicas e valor para o funcionamento do sistema. O grupo priorizou primeiro a arquitetura e o backend, pois o Core era a base para autenticar usuários, cadastrar entidades principais, aplicar regras de negócio e receber dados das coletas.

A ordem geral de priorização seguiu esta lógica:

1. Estrutura inicial do projeto e ambiente com Docker.
2. Modelagem do domínio principal e regras de negócio.
3. Comunicação entre Core e microsserviço por RabbitMQ.
4. Funcionalidades essenciais de cadastro, autenticação e consulta.
5. Frontend para interação com usuários.
6. Auditoria, monitoramento, backup e documentação.

Essa priorização permitiu que o grupo reduzisse riscos técnicos primeiro, principalmente nas integrações entre serviços e na persistência dos dados.

7. Dificuldades Encontradas

Uma das principais dificuldades foi a comunicação, pois todos os integrantes tinham outros afazeres acadêmicos, pessoais e profissionais. Como nem sempre era possível reunir todo o grupo em horários fixos, algumas decisões precisaram ser tomadas de forma assíncrona.

Outra dificuldade foi a existência de vários repositórios. Como o Core, o microsserviço, o frontend e a observabilidade ficaram separados, nem todas as issues estavam centralizadas no mesmo quadro Kanban. Em um momento, issues do frontend não foram passadas corretamente para o Kanban principal do Core, o que causou duplicidade de trabalho: duas pessoas chegaram a atuar na mesma tarefa. O problema ocorreu apenas uma vez e foi corrigido pelo grupo por meio de alinhamento e redistribuição das atividades.

Também houve desafios técnicos relacionados à integração entre serviços, principalmente por envolver backend, microsserviço, mensageria, bancos diferentes, frontend e observabilidade. A equipe precisou ajustar o planejamento ao longo do desenvolvimento para lidar com essas dependências.

8. Como as Dificuldades Foram Superadas

As dificuldades de comunicação foram tratadas com alinhamentos mais objetivos pelo WhatsApp e com reuniões on-line ou presenciais quando surgiam decisões importantes. O grupo passou a discutir com mais clareza quem ficaria responsável por cada tarefa antes de iniciar o desenvolvimento.

A duplicidade causada pelos repositórios separados foi corrigida com revisão do quadro e conversa entre os integrantes. A partir disso, o grupo teve mais cuidado ao verificar se uma tarefa já estava atribuída ou em andamento antes de iniciar uma nova implementação.

As dificuldades técnicas foram superadas por entregas incrementais. Em vez de tentar finalizar todo o sistema de uma vez, o grupo priorizou partes estruturais primeiro, como backend, Docker, mensageria e modelos de dados. Depois avançou para frontend, auditoria, monitoramento, backup e documentação.

9. Métricas de Progresso

As métricas coletadas no histórico dos repositórios mostram que o desenvolvimento não ocorreu de forma totalmente uniforme. Houve semanas de baixa atividade, semanas sem commits e semanas de maior concentração de entregas. Os principais picos ocorreram durante fases de estruturação do backend, integração, infraestrutura, observabilidade, CI e documentação.

Em relação a commits por semana, o projeto teve maior intensidade nas semanas em que foram implementadas partes estruturais e integrações importantes. Também houve uma semana sem commits, indicando uma pausa no fluxo de desenvolvimento ou baixa atividade registrada no GitHub.

Em relação a issues, o repositório principal concentrou a maior parte das tarefas abertas e fechadas.

Sobre o ritmo de entrega: ele aumentou nas fases finais do projeto, principalmente quando o grupo passou a integrar funcionalidades, corrigir problemas, documentar endpoints, criar diagramas de arquitetura, implementar monitoramento e preparar a entrega final.

De forma geral, as métricas indicam um processo com evolução real do produto, mas com concentração de trabalho em alguns períodos e necessidade de melhorar o acompanhamento formal das tarefas.

Resumo das métricas

Indicador	Observação
Commits por semana	Irregular, com picos em fases de estruturação/integração e ao menos uma semana sem commits
Issues abertas x fechadas	Concentradas majoritariamente no repositório principal (Core)
Velocity	Crescente nas fases finais do projeto (integração, documentação e monitoramento)
Distribuição de trabalho	Concentrada em períodos específicos, sem uniformidade entre as semanas

10. Retrospectiva Final

Ao final do projeto, o grupo avaliou que a combinação entre Scrum e Kanban foi adequada para a realidade da equipe. O Kanban ajudou a visualizar o fluxo das tarefas, enquanto as práticas inspiradas em Scrum ajudaram na divisão de responsabilidades, revisão de prioridades e organização das entregas.

O principal ponto positivo foi a preocupação inicial com arquitetura. Priorizar o backend e as integrações permitiu construir uma base mais consistente para o restante do sistema. Outro ponto

positivo foi a capacidade do grupo de se adaptar quando surgiram dificuldades, como problemas de comunicação, repositórios separados e ajustes nas prioridades.

Como ponto negativo, o grupo identificou que a falta de reuniões fixas dificultou o acompanhamento constante do progresso. Além disso, a separação em vários repositórios exigia mais disciplina na gestão das issues e do quadro Kanban. A duplicidade pontual de tarefa mostrou que o processo precisava de melhor sincronização entre os repositórios.

Recomendações para projetos futuros

- Definir uma reunião curta semanal, mesmo que on-line;
- Centralizar todas as tasks em um único board;
- Associar cada pull request a uma issue;
- Fechar issues no momento em que a funcionalidade for entregue;
- Revisar o Kanban periodicamente;
- Manter uma documentação mínima de decisões técnicas;
- Distribuir melhor o conhecimento entre os integrantes para reduzir dependência de uma única pessoa por módulo.

11. Conclusão

O processo de desenvolvimento do Coleta Premiada refletiu a realidade do grupo: uma metodologia híbrida, flexível e adaptada à disponibilidade dos integrantes. Apesar de não seguir Scrum de forma formal, o grupo utilizou práticas importantes de planejamento, divisão de tarefas, revisão e acompanhamento visual por Kanban.

As principais dificuldades estiveram relacionadas à comunicação, disponibilidade e organização entre repositórios. Ainda assim, o grupo conseguiu evoluir o sistema de forma incremental, entregando backend, microsserviço, frontend, observabilidade, auditoria, backup e documentação.

Como aprendizado final, o grupo percebeu que o uso de um quadro Kanban é essencial, mas precisa estar sempre atualizado e centralizado para evitar duplicidade, perda de rastreabilidade e divergência entre o trabalho realizado e o trabalho registrado.